

Pontifícia Universidade Católica de Goiás

Escola Politécnica e de Artes
Curso de Ciência da Computação

Atividade Final

Paradigmas de Linguagens de Programação
Sistema de Análise de Desempenho Acadêmico

Alunos:

Amanda Barbosa Ferreira Agapito
Guilherme José da Silva
Matheus Silva Pains

Professor: Roussian de Ramos Alves Gaioso

Goiânia, Goiás
12 de Junho de 2026

Sumário

1	Descrição do Problema	2
1.1	Dados de Entrada	2
1.2	Resultado Esperado	2
2	Linguagens Escolhidas	2
3	Parte 1 — Implementação Imperativa (Python)	3
3.1	Saída da Execução	4
4	Parte 2 — Implementação Orientada a Objetos (Python)	5
4.1	Saída da Execução	7
5	Parte 3 — Implementação Funcional (Python)	8
5.1	Saída da Execução	10
6	Parte 4 — Implementação Lógica (Prolog)	10
6.1	Exemplo de Consultas e Saída	12
7	Comparação de Legibilidade	12
8	Comparação de Facilidade de Escrita	13
9	Comparação de Confiabilidade	13
10	Comparação sobre Nomes, Escopo e Tempo de Vida	13
10.1	Versão Imperativa	13
10.2	Versão Orientada a Objetos	14
10.3	Versão Funcional	14
10.4	Versão Lógica (Prolog)	15
11	Comparação sobre Tipos de Dados	15
12	Comparação entre os Paradigmas	15
13	Conclusão	16

1 Descrição do Problema

O sistema recebe dados de alunos contendo nome, matrícula, três notas e frequência. A média é calculada como:

$$\text{média} = \frac{\text{nota1} + \text{nota2} + \text{nota3}}{3}$$

O aluno é **aprovado** se média $\geq 6,0$ e frequência $\geq 75\%$. Caso contrário, é **reprovado**.

1.1 Dados de Entrada

```
Ana;2024001;8.0;7.5;9.0;85
Bruno;2024002;5.0;6.0;4.5;80
Carlos;2024003;7.0;6.5;8.0;60
Daniela;2024004;9.0;9.5;10.0;95
Eduardo;2024005;3.0;4.0;2.5;70
Fernanda;2024006;6.0;6.0;6.0;75
```

1.2 Resultado Esperado

Nome	Matrícula	N1	N2	N3	Média	Freq	Situação
Ana	2024001	8,0	7,5	9,0	8,17	85	Aprovado
Bruno	2024002	5,0	6,0	4,5	5,17	80	Reprovado
Carlos	2024003	7,0	6,5	8,0	7,17	60	Reprovado
Daniela	2024004	9,0	9,5	10,0	9,50	95	Aprovado
Eduardo	2024005	3,0	4,0	2,5	3,17	70	Reprovado
Fernanda	2024006	6,0	6,0	6,0	6,00	75	Aprovado

2 Linguagens Escolhidas

- **Python (imperativo)** — permite demonstrar o paradigma procedural com variáveis, laços `while`, condicionais e funções, sem uso de classes ou funções de alta ordem.
- **Python (orientado a objetos)** — suporta classes com encapsulamento (`name mangling`), herança e métodos, sendo uma das linguagens OO mais utilizadas.
- **Python (funcional)** — oferece `map`, `filter`, `reduce`, lambdas e dicionários imutáveis, permitindo um estilo predominantemente funcional.
- **Prolog (lógico)** — linguagem declarativa por excelência, baseada em fatos e regras, ideal para demonstrar o paradigma lógico.

A escolha de Python para três paradigmas permite uma comparação mais justa, isolando o efeito do paradigma sem a interferência de diferenças sintáticas entre linguagens distintas. Prolog foi mantido por ser a referência natural do paradigma lógico.

3 Parte 1 — Implementação Imperativa (Python)

```
1 # =====
2 # Parte 1 -- Implementação Imperativa
3 # Sistema de Análise de Desempenho Acadêmico
4 # Paradigma: Imperativo/Procedural (Python)
5 # =====
6
7 import os
8
9 # ----- Funções procedurais -----
10
11 def ler_arquivo(caminho):
12     """Lê o arquivo CSV e retorna uma lista de listas com os dados."""
13     alunos = []
14     arquivo = open(caminho, "r", encoding="utf-8")
15     linhas = arquivo.readlines()
16     arquivo.close()
17
18     i = 0
19     while i < len(linhas):
20         linha = linhas[i].strip()
21         if linha != "":
22             partes = linha.split(";")
23             nome = partes[0]
24             matricula = partes[1]
25             nota1 = float(partes[2])
26             nota2 = float(partes[3])
27             nota3 = float(partes[4])
28             frequencia = int(partes[5])
29             alunos.append([nome, matricula, nota1, nota2, nota3,
30                             frequencia])
31             i = i + 1
32
33     return alunos
34
35 def calcular_media(nota1, nota2, nota3):
36     """Calcula a média aritmética de três notas."""
37     media = (nota1 + nota2 + nota3) / 3
38     return media
39
40
41 def verificar_aprovacao(media, frequencia):
42     """Retorna 'Aprovado' ou 'Reprovado' com base na média e frequência.
43     """
44
45     if media >= 6.0 and frequencia >= 75:
46         situacao = "Aprovado"
47     else:
48         situacao = "Reprovado"
49     return situacao
50
51 def gerar_relatorio(alunos):
```

```

51     """Percorre a lista de alunos, calcula médias e imprime o relatório.
52     """
53     print("=" * 70)
54     print("RELATÓRIO DE DESEMPENHO ACADÊMICO -- VERSÃO IMPERATIVA")
55     print("=" * 70)
56     print(f"{'Nome':<12} {'Matrícula':<12} {'N1':<6} {'N2':<6} {'N3':<6} {'Média':<8} {'Freq':<6} {'Situação'}")
57     print("-" * 70)
58     total_aprovados = 0
59     total_reprovados = 0
60
61     i = 0
62     while i < len(alunos):
63         nome = alunos[i][0]
64         matricula = alunos[i][1]
65         nota1 = alunos[i][2]
66         nota2 = alunos[i][3]
67         nota3 = alunos[i][4]
68         frequencia = alunos[i][5]
69
70         media = calcular_media(nota1, nota2, nota3)
71         situacao = verificar_aprovacao(media, frequencia)
72
73         if situacao == "Aprovado":
74             total_aprovados = total_aprovados + 1
75         else:
76             total_reprovados = total_reprovados + 1
77
78         print(f"{nome:<12} {matricula:<12} {nota1:<6.1f} {nota2:<6.1f} {nota3:<6.1f} {media:<8.2f} {frequencia:<6} {situacao}")
79
80         i = i + 1
81
82     print("-" * 70)
83     print(f"Total de alunos: {len(alunos)}")
84     print(f"Aprovados: {total_aprovados}")
85     print(f"Reprovados: {total_reprovados}")
86     print("=" * 70)
87
88     # ----- Programa principal -----
89
90     caminho_arquivo = os.path.join(os.path.dirname(os.path.abspath(__file__)), "alunos.csv")
91     lista_alunos = ler_arquivo(caminho_arquivo)
92     gerar_relatorio(lista_alunos)
93

```

Listing 1: parte1_imperativo.py

3.1 Saída da Execução

```

=====
RELATÓRIO DE DESEMPENHO ACADÊMICO -- VERSÃO IMPERATIVA
=====

```

Nome	Matricula	N1	N2	N3	Média	Freq	Situação
Ana	2024001	8.0	7.5	9.0	8.17	85	Aprovado
Bruno	2024002	5.0	6.0	4.5	5.17	80	Reprovado
Carlos	2024003	7.0	6.5	8.0	7.17	60	Reprovado
Daniela	2024004	9.0	9.5	10.0	9.50	95	Aprovado
Eduardo	2024005	3.0	4.0	2.5	3.17	70	Reprovado
Fernanda	2024006	6.0	6.0	6.0	6.00	75	Aprovado

Total de alunos: 6							
Aprovados: 3							
Reprovados: 3							
=====							

4 Parte 2 — Implementação Orientada a Objetos (Python)

```

1  # =====
2  # Parte 2 -- Implementação Orientada a Objetos
3  # Sistema de Análise de Desempenho Acadêmico
4  # Paradigma: Orientado a Objetos (Python)
5  # =====
6
7  import os
8
9
10 class Aluno:
11     """Representa um aluno com seus dados acadêmicos."""
12
13     def __init__(self, nome, matricula, nota1, nota2, nota3, frequencia)
14         :
15         self.__nome = nome
16         self.__matricula = matricula
17         self.__nota1 = nota1
18         self.__nota2 = nota2
19         self.__nota3 = nota3
20         self.__frequencia = frequencia
21
22     # ----- Getters (encapsulamento) -----
23
24     def get_nome(self):
25         return self.__nome
26
27     def get_matricula(self):
28         return self.__matricula
29
30     def get_nota1(self):
31         return self.__nota1
32
33     def get_nota2(self):
34         return self.__nota2
35
36     def get_nota3(self):
37         return self.__nota3
38
39     def get_frequencia(self):
40         return self.__frequencia

```

```

40
41 # ----- Métodos de negócio -----
42
43 def calcular_media(self):
44     """Calcula e retorna a média aritmética das três notas."""
45     return (self.__nota1 + self.__nota2 + self.__nota3) / 3
46
47 def verificar_aprovacao(self):
48     """Retorna 'Aprovado' se média >= 6.0 e frequência >= 75, senão
49         'Reprovado'."""
50     media = self.calcular_media()
51     if media >= 6.0 and self.__frequencia >= 75:
52         return "Aprovado"
53     return "Reprovado"
54
55 def __str__(self):
56     media = self.calcular_media()
57     situacao = self.verificar_aprovacao()
58     return (f"{self.__nome:<12} {self.__matricula:<12} "
59             f"{self.__nota1:<6.1f} {self.__nota2:<6.1f} {self.
60                 __nota3:<6.1f} "
61             f"{media:<8.2f} {self.__frequencia:<6} {situacao}")
62
63 class Turma:
64     """Gerencia uma coleção de alunos e gera relatórios."""
65
66     def __init__(self):
67         self.__alunos = []
68
69     def carregar_de_arquivo(self, caminho):
70         """Lê o arquivo CSV e popula a lista de alunos."""
71         with open(caminho, "r", encoding="utf-8") as arquivo:
72             for linha in arquivo:
73                 linha = linha.strip()
74                 if linha:
75                     partes = linha.split(";")
76                     aluno = Aluno(
77                         nome=partes[0],
78                         matricula=partes[1],
79                         nota1=float(partes[2]),
80                         nota2=float(partes[3]),
81                         nota3=float(partes[4]),
82                         frequencia=int(partes[5])
83                     )
84                     self.__alunos.append(aluno)
85
86     def obter_aprovados(self):
87         """Retorna a lista de alunos aprovados."""
88         aprovados = []
89         for aluno in self.__alunos:
90             if aluno.verificar_aprovacao() == "Aprovado":
91                 aprovados.append(aluno)
92         return aprovados
93
94     def obter_reprovados(self):

```

```

94     """Retorna a lista de alunos reprovados."""
95     reprovados = []
96     for aluno in self.__alunos:
97         if aluno.verificar_aprovacao() == "Reprovado":
98             reprovados.append(aluno)
99     return reprovados
100
101     def gerar_relatorio(self):
102         """Imprime o relatório completo de desempenho."""
103         print("=" * 70)
104         print("RELATÓRIO DE DESEMPENHO ACADÊMICO -- VERSÃO ORIENTADA A
105             OBJETOS")
106         print("=" * 70)
107         print(f"{'Nome':<12} {'Matrícula':<12} {'N1':<6} {'N2':<6} {'N3
108             ':<6} {'Média':<8} {'Freq':<6} {'Situação'}")
109         print("-" * 70)
110
111         for aluno in self.__alunos:
112             print(aluno)
113
114         aprovados = self.obter_aprovados()
115         reprovados = self.obter_reprovados()
116
117         print("-" * 70)
118         print(f"Total de alunos: {len(self.__alunos)}")
119         print(f"Aprovados: {len(aprovados)}")
120         print(f"Reprovados: {len(reprovados)}")
121         print("=" * 70)
122
123     # ----- Programa principal -----
124     caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
125         alunos.csv")
126     turma = Turma()
127     turma.carregar_de_arquivo(caminho)
128     turma.gerar_relatorio()

```

Listing 2: parte2_oo.py

4.1 Saída da Execução

```

=====
RELATÓRIO DE DESEMPENHO ACADÊMICO -- VERSÃO ORIENTADA A OBJETOS
=====
Nome           Matrícula    N1      N2      N3      Média    Freq    Situação
-----
Ana            2024001      8.0     7.5     9.0     8.17     85      Aprovado
Bruno          2024002      5.0     6.0     4.5     5.17     80      Reprovado
Carlos         2024003      7.0     6.5     8.0     7.17     60      Reprovado
Daniela        2024004      9.0     9.5     10.0    9.50     95      Aprovado
Eduardo        2024005      3.0     4.0     2.5     3.17     70      Reprovado
Fernanda       2024006      6.0     6.0     6.0     6.00     75      Aprovado
-----
Total de alunos: 6
Aprovados: 3
Reprovados: 3
=====

```


5 Parte 3 — Implementação Funcional (Python)

```
1 # =====
2 # Parte 3 -- Implementação Funcional
3 # Sistema de Análise de Desempenho Acadêmico
4 # Paradigma: Funcional (Python)
5 # =====
6
7 import os
8 from functools import reduce
9
10
11 # ----- Funções puras -----
12
13 def parse_linha(linha):
14     """Converte uma linha CSV em um dicionário imutável de dados do
15     aluno."""
16     partes = linha.strip().split(";")
17     return {
18         "nome": partes[0],
19         "matricula": partes[1],
20         "nota1": float(partes[2]),
21         "nota2": float(partes[3]),
22         "nota3": float(partes[4]),
23         "frequencia": int(partes[5])
24     }
25
26 def calcular_media(aluno):
27     """Função pura: retorna a média aritmética das três notas."""
28     return (aluno["nota1"] + aluno["nota2"] + aluno["nota3"]) / 3
29
30
31 def verificar_aprovacao(aluno):
32     """Função pura: retorna True se o aluno está aprovado."""
33     return calcular_media(aluno) >= 6.0 and aluno["frequencia"] >= 75
34
35
36 def gerar_resultado(aluno):
37     """Função pura: retorna um novo dicionário com média e situação
38     adicionados."""
39     media = calcular_media(aluno)
40     situacao = "Aprovado" if verificar_aprovacao(aluno) else "Reprovado"
41     return {**aluno, "media": media, "situacao": situacao}
42
43 def contar_por_situacao(accumulator, resultado):
44     """Função para reduce: conta aprovados e reprovados."""
45     if resultado["situacao"] == "Aprovado":
46         return (accumulator[0] + 1, accumulator[1])
47     else:
48         return (accumulator[0], accumulator[1] + 1)
49
50
51 # ----- Entrada -----
```

```

52
53 def ler_dados(caminho):
54     """Lê o arquivo e retorna lista de dicionários (separação entrada/
55         processamento)."""
56     with open(caminho, "r", encoding="utf-8") as arquivo:
57         linhas = arquivo.readlines()
58         return list(map(parse_linha, filter(lambda l: l.strip() != "",
59             linhas)))
59
60 # ----- Saída -----
61
62 def imprimir_relatorio(resultados):
63     """Imprime o relatório formatado (separação de saída)."""
64     print("=" * 70)
65     print("RELATÓRIO DE DESEMPENHO ACADÊMICO -- VERSÃO FUNCIONAL")
66     print("=" * 70)
67     print(f"{'Nome':<12} {'Matrícula':<12} {'N1':<6} {'N2':<6} {'N3':<6} "
68         f"{'Média':<8} {'Freq':<6} {'Situação'}")
69     print("-" * 70)
70
71     for r in resultados:
72         print(f"{r['nome']:<12} {r['matricula']:<12} "
73             f"{r['nota1']:<6.1f} {r['nota2']:<6.1f} {r['nota3']:<6.1f} "
74             f"{r['media']:<8.2f} {r['frequencia']:<6} {r['situacao']}"
75             )
76
77     aprovados, reprovados = reduce(contar_por_situacao, resultados, (0,
78         0))
79
80     print("-" * 70)
81     print(f"Total de alunos: {len(resultados)}")
82     print(f"Aprovados: {aprovados}")
83     print(f"Reprovados: {reprovados}")
84     print("=" * 70)
85
86 # ----- Pipeline funcional -----
87
88 caminho = os.path.join(os.path.dirname(os.path.abspath(__file__)), "
89     alunos.csv")
90
91 # Entrada
92 alunos = ler_dados(caminho)
93
94 # Processamento (map aplica gerar_resultado a cada aluno)
95 resultados = list(map(gerar_resultado, alunos))
96
97 # Exemplo de filter: listar apenas aprovados
98 aprovados = list(filter(lambda r: r["situacao"] == "Aprovado",
99     resultados))
100
101 # Saída
102 imprimir_relatorio(resultados)

```

```

100 print("\n--- Alunos Aprovados (via filter) ---")
101 for a in aprovados:
102     print(f"    {a['nome']} -- Média: {a['media']:.2f}")

```

Listing 3: parte3_funcional.py

5.1 Saída da Execução

```

=====
RELATÓRIO DE DESEMPENHO ACADÊMICO -- VERSÃO FUNCIONAL
=====
Nome           Matrícula    N1      N2      N3      Média    Freq    Situação
-----
Ana            2024001      8.0     7.5     9.0     8.17     85     Aprovado
Bruno          2024002      5.0     6.0     4.5     5.17     80     Reprovado
Carlos         2024003      7.0     6.5     8.0     7.17     60     Reprovado
Daniela        2024004      9.0     9.5     10.0    9.50     95     Aprovado
Eduardo        2024005      3.0     4.0     2.5     3.17     70     Reprovado
Fernanda       2024006      6.0     6.0     6.0     6.00     75     Aprovado
-----
Total de alunos: 6
Aprovados: 3
Reprovados: 3
=====

--- Alunos Aprovados (via filter) ---
Ana -- Média: 8.17
Daniela -- Média: 9.50
Fernanda -- Média: 6.00

```

6 Parte 4 — Implementação Lógica (Prolog)

```

1  % =====
2  % Parte 4 -- Implementação Lógica
3  % Sistema de Análise de Desempenho Acadêmico
4  % Paradigma: Lógico/Declarativo (Prolog)
5  % =====
6
7  % ----- Fatos: aluno(Nome, Matrícula, Nota1, Nota2, Nota3, Frequencia)
8  % -----
9
10 aluno(ana,      2024001, 8.0, 7.5, 9.0, 85).
11 aluno(bruno,    2024002, 5.0, 6.0, 4.5, 80).
12 aluno(carlos,   2024003, 7.0, 6.5, 8.0, 60).
13 aluno(daniela,  2024004, 9.0, 9.5, 10.0, 95).
14 aluno(eduardo,  2024005, 3.0, 4.0, 2.5, 70).
15 aluno(fernanda, 2024006, 6.0, 6.0, 6.0, 75).
16
17 % ----- Regra: calcular média -----
18
19 media(Nome, Media) :-
20     aluno(Nome, _, N1, N2, N3, _),
21     Media is (N1 + N2 + N3) / 3.
22
23 % ----- Regra: verificar frequência suficiente -----

```

```

24 frequencia_suficiente(Nome) :-
25     aluno(Nome, _, _, _, Freq),
26     Freq >= 75.
27
28 % ----- Regra: determinar aprovação -----
29
30 aprovado(Nome) :-
31     media(Nome, Media),
32     Media >= 6.0,
33     frequencia_suficiente(Nome).
34
35 % ----- Regra: determinar reprovação -----
36
37 reprovado(Nome) :-
38     aluno(Nome, _, _, _, _),
39     \+ aprovado(Nome).
40
41 % ----- Regra: exibir situação de um aluno -----
42
43 situacao(Nome, Matricula, Media, Freq, Situacao) :-
44     aluno(Nome, Matricula, _, _, _, Freq),
45     media(Nome, Media),
46     (aprovado(Nome) -> Situacao = aprovado ; Situacao = reprovado).
47
48 % ----- Consultas úteis -----
49 % Para consultar no interpretador Prolog:
50 %
51 % Listar todos os aprovados:
52 %     ?- aprovado(X).
53 %
54 % Listar todos os reprovados:
55 %     ?- reprovado(X).
56 %
57 % Ver situação de um aluno:
58 %     ?- situacao(ana, Mat, Media, Freq, Sit).
59 %
60 % Ver situação de todos:
61 %     ?- situacao(Nome, Mat, Media, Freq, Sit).
62
63 % ----- Regra para imprimir relatório -----
64
65 imprimir_aluno(Nome) :-
66     situacao(Nome, Matricula, Media, Freq, Situacao),
67     format("~w | ~w | Media: ~2f | Freq: ~w | ~w~n",
68           [Nome, Matricula, Media, Freq, Situacao]).
69
70 relatorio :-
71     write('====='), nl,
72     write('RELATORIO - VERSAO LOGICA (PROLOG)'), nl,
73     write('====='), nl,
74     forall(aluno(Nome, _, _, _, _), imprimir_aluno(Nome)),
75     write('====='), nl.

```

Listing 4: parte4_logico.pl

6.1 Exemplo de Consultas e Saída

```
?- aprovado(X).
X = ana ;
X = daniela ;
X = fernanda.

?- reprovado(X).
X = bruno ;
X = carlos ;
X = eduardo.

?- situacao(ana, Mat, Media, Freq, Sit).
Mat = 2024001, Media = 8.166666666666666, Freq = 85, Sit = aprovado.

?- relatorio.
=====
RELATORIO - VERSAO LOGICA (PROLOG)
=====
ana | 2024001 | Media: 8.17 | Freq: 85 | aprovado
bruno | 2024002 | Media: 5.17 | Freq: 80 | reprovado
carlos | 2024003 | Media: 7.17 | Freq: 60 | reprovado
daniela | 2024004 | Media: 9.50 | Freq: 95 | aprovado
eduardo | 2024005 | Media: 3.17 | Freq: 70 | reprovado
fernanda | 2024006 | Media: 6.00 | Freq: 75 | aprovado
=====
true.
```

7 Comparação de Legibilidade

Critério	Análise
Clareza dos nomes	Todas as versões Python usam nomes descritivos em português. Prolog usa átomos curtos (ana , aprovado), concisos mas claros.
Organização	A versão OO é a mais organizada: dados e lógica ficam encapsulados em classes. A funcional separa bem entrada/processamento/saída. A imperativa é linear e direta.
Detalhes sintáticos	Prolog tem a menor quantidade de sintaxe. Python imperativo exige mais linhas por usar while e índices explícitos.
Fluxo do programa	A imperativa é a mais fácil de seguir passo a passo. A funcional exige entender composição de funções. Prolog não tem fluxo explícito.
Localização das regras	Em Prolog, cada regra é uma cláusula isolada e fácil de encontrar. Na OO, estão nos métodos da classe. Na funcional, em funções nomeadas.

Tabela 1: Comparação de legibilidade entre os paradigmas.

Conclusão: Para leitores iniciantes, a versão imperativa é a mais legível. Para projetos maiores, a OO oferece melhor organização. Prolog é o mais conciso, mas exige familiaridade com o paradigma.

8 Comparação de Facilidade de Escrita

Critério	Análise
Quantidade de código	Prolog: ~40 linhas. Funcional: ~70 linhas. Imperativa: ~80 linhas. OO: ~100 linhas.
Dificuldade sintática	Python é uniforme nos três paradigmas. Prolog exige aprender unificação e backtracking.
Manipulação de listas	A funcional é a mais natural com <code>map/filter</code> . A imperativa usa índices manuais. A OO itera com <code>for</code> .
Estruturas auxiliares	A OO exige criar classes. A funcional usa dicionários. A imperativa usa listas de listas. Prolog usa fatos diretamente.
Bibliotecas	Apenas a funcional importa <code>functools.reduce</code> . As demais usam apenas a biblioteca padrão.

Tabela 2: Comparação de facilidade de escrita.

Conclusão: A versão funcional Python foi a mais rápida de implementar pela expressividade de `map/filter`. Prolog é extremamente conciso, mas exige mudança de mentalidade.

9 Comparação de Confiabilidade

Conclusão: A versão funcional oferece maior confiabilidade por usar funções puras. A OO vem em segundo lugar pelo encapsulamento. Python e Prolog são mais seguros que C por não permitirem acesso direto à memória.

10 Comparação sobre Nomes, Escopo e Tempo de Vida

10.1 Versão Imperativa

- **Nomes declarados:** `caminho_arquivo`, `lista_alunos` (globais); `nome`, `media`, `i` (locais em funções).
- **Visibilidade:** As variáveis globais são visíveis em todo o módulo. As locais existem apenas dentro de suas funções.
- **Escopo:** Funções como `calcular_media()` têm escopo local; parâmetros e variáveis internas não vazam.
- **Tempo de vida:** Variáveis locais são criadas na chamada da função e destruídas ao retornar. As globais existem durante toda a execução.

Critério	Análise
Verificação de tipos	Python tem tipagem dinâmica — erros de tipo só aparecem em tempo de execução. Prolog não tem tipos explícitos.
Erros em tempo de execução	A imperativa é a mais suscetível a erros de índice (<code>alunos[i][5]</code>). A OO protege dados com encapsulamento. A funcional, ao usar funções puras, reduz efeitos colaterais.
Entrada inválida	Nenhuma versão trata entradas inválidas de forma robusta. Seria necessário adicionar <code>try/except</code> em Python ou validação em Prolog.
Acesso à memória	Python e Prolog não permitem acesso direto à memória, eliminando riscos de buffer overflow (diferente de C).
Testabilidade	A funcional é a mais testável: funções puras sem efeito colateral podem ser testadas isoladamente. A OO permite testes unitários por método.

Tabela 3: Comparação de confiabilidade.

10.2 Versão Orientada a Objetos

- **Nomes declarados:** Classes `Aluno`, `Turma`; atributos `self.__nome`, `self.__nota1`; métodos `calcular_media()`.
- **Visibilidade:** Atributos com `__` (name mangling) são visíveis apenas dentro da classe. Métodos públicos são acessíveis externamente.
- **Escopo:** Cada objeto tem seu próprio escopo de atributos. Métodos acessam `self` para dados da instância.
- **Tempo de vida:** Objetos existem enquanto houver referência a eles. O garbage collector de Python os destrói quando não há mais referências.

10.3 Versão Funcional

- **Nomes declarados:** Funções puras `calcular_media()`, `verificar_aprovacao()`, `gerar_resultado()`; variável `alunos` no nível do módulo.
- **Visibilidade:** Cada função vê apenas seus parâmetros e variáveis locais. Não há estado mutável compartilhado.
- **Escopo:** Funções são de escopo global. Lambdas em `filter()` capturam variáveis do escopo envolvente (closure).
- **Tempo de vida:** Dados são criados a cada chamada e descartados após o retorno. Listas intermediárias (`resultados`) vivem no escopo do módulo.

10.4 Versão Lógica (Prolog)

- **Nomes declarados:** Predicados `aluno/6`, `media/2`, `aprovado/1`; variáveis lógicas `Nome`, `Media`.
- **Visibilidade:** Fatos e regras são globais no programa. Variáveis lógicas existem apenas dentro de sua cláusula.
- **Escopo:** Cada cláusula é independente. A variável `Nome` em `aprovado(Nome)` só existe naquela regra.
- **Tempo de vida:** Variáveis são instanciadas durante a unificação e deixam de existir quando a consulta termina ou faz backtracking.

11 Comparação sobre Tipos de Dados

Aspecto	Python (3 versões)	Prolog
Tipagem	Dinâmica, forte	Sem tipos explícitos
Tipos primitivos	<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>	Átomos, números, variáveis
Strings	Tipo nativo <code>str</code> , imutável	Átomos (<code>ana</code> , <code>bruno</code>)
Coleções	Listas, dicionários, tuplas	Listas Prolog, termos compostos
Objetos	Sim (versão OO)	Não existe o conceito
Conversão de tipo	Explícita: <code>float()</code> , <code>int()</code>	Automática via <code>is</code>
Erros de tipo	<code>TypeError</code> em tempo de execução	Falha na unificação

Tabela 4: Comparação de tipos de dados.

12 Comparação entre os Paradigmas

1. Como a versão imperativa organiza a solução?

Sequencialmente: lê dados, percorre com laço `while`, calcula média, verifica condição, imprime. O programador controla cada passo.

2. Como a versão orientada a objetos organiza a solução?

Modela o domínio: `Aluno` encapsula dados e comportamento, `Turma` gerencia a coleção. A lógica é distribuída em métodos.

3. Como a versão funcional organiza a solução?

Como pipeline de transformações: `ler_dados` $\rightarrow$ `map(gerar_resultado)` $\rightarrow$ `imprimir_relatorio`
Dados fluem entre funções puras.

4. **Como a versão declarativa organiza a solução?**

Declara fatos (dados dos alunos) e regras (média, aprovação). O motor de inferência de Prolog encontra as respostas.

5. **Qual versão ficou mais próxima da forma humana?**

Prolog. A regra `aprovado(Nome) :- media(Nome, M), M >= 6.0, ...` é quase linguagem natural.

6. **Qual versão ficou mais próxima da máquina?**

A imperativa. O laço `while` com índice manual e atribuições explícitas reflete como o processador executa instruções.

7. **Qual versão parece mais adequada para esse problema?**

A funcional. O problema é essencialmente uma transformação de dados (entrada $\rightarrow$ processamento $\rightarrow$ saída), que é o caso de uso ideal do paradigma funcional.

13 Conclusão

Cada paradigma demonstrou forças e limitações claras ao resolver o mesmo problema:

- O **paradigma imperativo** é o mais intuitivo para iniciantes e oferece controle total sobre o fluxo, mas gera código mais verboso e propenso a erros de índice.
- O **paradigma orientado a objetos** brilha em organização e manutenibilidade. Encapsulamento protege os dados e facilita extensões futuras (ex.: adicionar novos tipos de aluno). É o mais adequado para sistemas grandes.
- O **paradigma funcional** produziu o código mais conciso e testável. Funções puras sem efeito colateral facilitam depuração e paralelismo. É ideal para problemas de transformação de dados.
- O **paradigma lógico** é o mais declarativo e conciso. O programador descreve *o que* quer, não *como* fazer. É poderoso para regras de negócio e sistemas especialistas, mas limitado para I/O e interfaces.

Não existe paradigma universalmente “melhor”. A escolha depende do problema: sistemas corporativos favorecem OO; pipelines de dados favorecem funcional; regras de inferência favorecem lógico; scripts simples favorecem imperativo. O programador competente domina múltiplos paradigmas e escolhe o mais adequado para cada situação.